SUBMITTED BY- KISHAN KUMAR BADHAI
ENROLL- GGV/24/05132
ROLL- 24070132
COURSE- Bsc.CS(HONS)
SEM- 4TH

1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

```
import java.io.*;
import java.util.*;

class Farmer {
    String name;
    String fruit;
    String city;
    int quantity;


    public Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }

    @Override
    public String toString() {
        return String.format("%-20s %-15s %-15s %d", name, fruit, city,
quantity);
    }
}

public class FarmerManagement {
    static List<Farmer> farmers = new ArrayList<>();

    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter details for 10 farmers:");
        for (int i = 0; i < 10; i++) {
            System.out.print("Farmer " + (i + 1) + " - Name: ");
            String name = sc.nextLine();
            System.out.print("Fruit: ");
            String fruit = sc.nextLine();
            System.out.print("City: ");
            String city = sc.nextLine();
            System.out.print("Quantity: ");
```

```java
            int quantity = Integer.parseInt(sc.nextLine());

            farmers.add(new Farmer(name, fruit, city, quantity));
        }

        saveToFile("before_sorting.txt", false);

        farmers.sort(Comparator.comparingInt(f -> f.quantity));

        saveToFile("after_sorting.txt", false);

        System.out.print("\nSearch by farmer name: ");
        String searchName = sc.nextLine();
        searchFarmer(searchName);

        sc.close();
    }

    static void saveToFile(String filename, boolean append) throws IOException
{
        FileWriter fw = new FileWriter(filename, append);
        fw.write(String.format("%-20s %-15s %-15s %s\n", "Name", "Fruit",
"City", "Quantity"));
        fw.write("=".repeat(65) + "\n");
        for (Farmer f : farmers) {
            fw.write(f.toString() + "\n");
        }
        fw.write("\n");
        fw.close();
        System.out.println("Data saved to " + filename);
    }

    static void searchFarmer(String name) {
        boolean found = false;
        System.out.println("\n" + String.format("%-20s %-15s %-15s %s\n",
"Name", "Fruit", "City", "Quantity"));
        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(name)) {
                System.out.println(f);
                found = true;
            }
        }
        if (!found) {
            System.out.println("Farmer not found!");
        }
    }
}
```

OUTPUT –

```
Enter details for 10 farmers:
Farmer 1 - Name: RAMESH
Fruit: MANGO
City: RAIPUR
Quantity: 1000
Farmer 2 - Name: RAM
Fruit: MELON
City: INDORE
Quantity: 5000
Farmer 3 - Name: MOHAN
Fruit: BLUEBERRY
City: BHUBNESWAR
Quantity: 600
Farmer 4 - Name: HRUSHIKESH
Fruit: ORANGES
City: SAMBALPUR
Quantity: 700
Farmer 5 - Name: SHYAM
Fruit: PAPAYA
City: DELHI
Quantity: 1000
Farmer 6 - Name: DARSHAN
Fruit: GRAPES
City: BARGARH
Quantity: 2000
Farmer 7 - Name: RAMESH
Fruit: BANANA
City: JHARSUGUDA
Quantity: 3000
Farmer 8 - Name: BISWANATH
Fruit: AVACADO
City: ROURKELA
Quantity: 500
Farmer 9 - Name: RAVI
Fruit: APPLE
City: BILASPUR
Quantity: 2000
Farmer 10 - Name: HARI
Fruit: GUAVA
City: RAIPUR
Quantity: 500
Data saved to before_sorting.txt
Data saved to after_sorting.txt

Search by farmer name: RAVI

Name                    Fruit              City               Quantity

RAVI                    APPLE              BILASPUR           2000
```

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class FruitInventory {
    static class Fruit {
        String name;
        String quality;
        int quantity;

        Fruit(String name, String quality, int quantity) {
            this.name = name;
            this.quality = quality;
            this.quantity = quantity > 1000 ? quantity / 2 : quantity;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Map<String, Fruit> inventory = new HashMap<>();
```

```java

        String[] fruits = {"Mango", "Apple", "Papaya"};
        String[] qualities = {"Good", "Bad", "Average"};

        // Input fruits with quantity and quality
        for (String fruit : fruits) {
            System.out.println("Enter quantity for " + fruit + ":");
            int quantity = sc.nextInt();
            sc.nextLine();

            System.out.println("Enter quality (Good/Bad/Average):");
            String quality = sc.nextLine();

            if (isValidQuality(quality, qualities)) {
                inventory.put(fruit + "_" + quality, new Fruit(fruit, quality, quantity));
            } else {
                System.out.println("Invalid quality. Skipping " + fruit);
            }
        }

        // Search for fruit with given quality
        System.out.println("\nSearch for fruit:");
        System.out.println("Enter fruit name:");
        String searchFruit = sc.nextLine();
        System.out.println("Enter quality:");
        String searchQuality = sc.nextLine();

        searchFruit(inventory, searchFruit, searchQuality);
        sc.close();
    }

    static boolean isValidQuality(String quality, String[] qualities) {
        for (String q : qualities) {
            if (q.equalsIgnoreCase(quality)) return true;
        }
        return false;
    }

    static void searchFruit(Map<String, Fruit> inventory, String name, String quality) {
        Fruit result = inventory.get(name + "_" + quality);
        if (result != null) {
            System.out.println("Found: " + result.name + " (" + result.quality + ") - Quantity: " + result.quantity);
        } else {
            System.out.println("Fruit not found!");
```

```
        }
    }
}
```

OUTPUT-

```
Bad
Enter quantity for Papaya:
400
Enter quality (Good/Bad/Average):
Average

Search for fruit:
Enter fruit name:
Mango
Enter quality:
Good
Found: Mango (Good) - Quantity: 1000
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

The available orchards in a tree-view.

Search for a given fruit in the available orchards after taking input from the user & print the tree.

```java
import java.util.Scanner;
import java.util.HashMap;
import java.util.Map;

public class OrchardSearch {
    static class Orchard {
        String name;
        String fruit;
        String quality;

        Orchard(String name, String fruit, String quality) {
            this.name = name;
            this.fruit = fruit;
            this.quality = quality;
        }
    }

    public static void main(String[] args) {
        Map<String, Orchard> orchards = new HashMap<>();
```

```java
        orchards.put("A", new Orchard("A", "Apple", "Good"));
        orchards.put("B", new Orchard("B", "Litchi", "Bad"));
        orchards.put("C", new Orchard("C", "Mango", "Good"));
        orchards.put("D", new Orchard("D", "Guava", "Bad"));

        System.out.println("=== Orchard Tree ===");
        printOrchards(orchards);

        Scanner scanner = new Scanner(System.in);
        System.out.print("\nEnter fruit name to search: ");
        String searchFruit = scanner.nextLine().trim();

        System.out.println("\n=== Search Results for: " + searchFruit + "
===");
        boolean found = false;
        for (Orchard orchard : orchards.values()) {
            if (orchard.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println("Orchard " + orchard.name + " -> " +
orchard.fruit + " (Quality: " + orchard.quality + ")");
                found = true;
            }
        }
        if (!found) {
            System.out.println("Fruit not found in any orchard.");
        }
        scanner.close();
    }

    static void printOrchards(Map<String, Orchard> orchards) {
        for (String key : orchards.keySet()) {
            Orchard o = orchards.get(key);
            System.out.println("├─ Orchard " + o.name + " -> " + o.fruit + "
(Quality: " + o.quality + ")");
        }
    }
}
```

OUTPUT-

```
=== Orchard Tree ===
├─ Orchard A -> Apple (Quality: Good)
├─ Orchard B -> Litchi (Quality: Bad)
├─ Orchard C -> Mango (Quality: Good)
├─ Orchard D -> Guava (Quality: Bad)

Enter fruit name to search: Litchi

=== Search Results for: Litchi ===
Orchard B -> Litchi (Quality: Bad)
```

4. Take a map provided as a graph & display all the subtrees of connected points.

```java
import java.util.*;

public class MAP {

    // DFS to find one connected component
    static void dfs(int node, boolean[] visited, List<List<Integer>> graph,
List<Integer> component) {
        visited[node] = true;
        component.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph, component);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        // Adjacency list
        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u); // Undirected graph
        }
```

```java
        boolean[] visited = new boolean[n];

        System.out.println("\nSubtrees (Connected Components):");

        // Find all components
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                List<Integer> component = new ArrayList<>();
                dfs(i, visited, graph, component);

                // Print one subtree
                System.out.println(component);
            }
        }

        sc.close();
    }
}
```

OUTPUT-

```
Enter number of vertices: 4
Enter number of edges: 3
Enter edges (u v):
0 1
1 2
2 3

Subtrees (Connected Components):
[0, 1, 2, 3]
```

5.Enter entry point & goal point & produce the matching subtree of the graph.

```java
import java.util.*;

public class GRAPH{

    // DFS to find path
    static boolean dfs(int current, int goal, boolean[] visited,
                       List<List<Integer>> graph, List<Integer> path) {

        visited[current] = true;
        path.add(current);

        // If goal found
        if (current == goal) {
            return true;
        }
```

```java
        for (int neighbor : graph.get(current)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited, graph, path)) {
                    return true;
                }
            }
        }

        // Backtrack if not found
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u); // undirected
        }

        System.out.print("Enter entry point: ");
        int start = sc.nextInt();

        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();

        boolean[] visited = new boolean[n];
        List<Integer> path = new ArrayList<>();

        if (dfs(start, goal, visited, graph, path)) {
```

```
            System.out.println("\nMatching Subtree (Path):");

            // Print as tree/path
            for (int node : path) {
                System.out.print(node + " -> ");
            }
            System.out.println("Goal");
        } else {
            System.out.println("\nNo path exists between given points.");
        }

        sc.close();
    }
}
```

OUTPUT-

```
Enter number of vertices: 4
Enter number of edges: 3
Enter edges (u v):
0 1
1 2
2 3
Enter entry point: 0
Enter goal point: 2

Matching Subtree (Path):
0 -> 1 -> 2 -> Goal
```

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
 import java.util.*;

class Edge {
    int target, weight;

    Edge(int target, int weight) {
        this.target = target;
        this.weight = weight;
    }
}

public class WEIGHTED_GRAPH {

    static void dijkstra(List<List<Edge>> graph, int start, int end) {
        int n = graph.size();

        int[] dist = new int[n];
        int[] parent = new int[n];
```

```java
        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);

        dist[start] = 0;

        PriorityQueue<int[]> pq = new
PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
        pq.add(new int[]{start, 0});

        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0];
            int d = current[1];

            if (d > dist[node]) continue;

            for (Edge edge : graph.get(node)) {
                int newDist = dist[node] + edge.weight;

                if (newDist < dist[edge.target]) {
                    dist[edge.target] = newDist;
                    parent[edge.target] = node;
                    pq.add(new int[]{edge.target, newDist});
                }
            }
        }

        // If unreachable
        if (dist[end] == Integer.MAX_VALUE) {
            System.out.println("No path exists.");
            return;
        }

        // Reconstruct path
        List<Integer> path = new ArrayList<>();
        for (int v = end; v != -1; v = parent[v]) {
            path.add(v);
        }
        Collections.reverse(path);

        // Output
        System.out.println("\nShortest Path:");
        for (int node : path) {
            System.out.print(node + " -> ");
        }
        System.out.println("End");
```

```java
        System.out.println("Total Cost: " + dist[end]);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Edge>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(new Edge(v, w));
            graph.get(v).add(new Edge(u, w)); // undirected
        }

        System.out.print("Enter start node: ");
        int start = sc.nextInt();

        System.out.print("Enter end node: ");
        int end = sc.nextInt();

        dijkstra(graph, start, end);

        sc.close();
    }
}
```

OUTPUT-

```
Enter number of vertices: 3
Enter number of edges: 3
Enter edges (u v weight):
0 1 5
1 2 2
0 2 10
Enter start node: 0
Enter end node: 2

Shortest Path:
0 -> 1 -> 2 -> End
Total Cost: 7
```

7. Write a program to demonstrate the use of all keywords made for exception handling

```java
import java.io.IOException;

public class ExceptionKeywords {

    // Using 'throws' to declare that this method might cause an error
    public static void checkFileStatus(boolean isFileMissing) throws
IOException {
        if (isFileMissing) {
            // Using 'throw' to manually trigger an exception
            throw new IOException("File not found on system!");
        } else {
            System.out.println("File is present.");
        }
    }

    public static void main(String[] args) {
        // Using 'try' to monitor for exceptions
        try {
            System.out.println("Attempting to access file...");
            checkFileStatus(true);

        }
        // Using 'catch' to handle the specific error thrown
        catch (IOException e) {
            System.out.println("Caught an Exception: " + e.getMessage());
        }
        // Using 'finally' for cleanup code that must run
        finally {
            System.out.println("Cleanup: Closing all file streams.");
        }

        System.out.println("Program continues normally...");
    }
}
```

OUTPUT-

```
Attempting to access file...
Caught an Exception: File not found on system!
Cleanup: Closing all file streams.
Program continues normally...
```

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```java
import java.util.Scanner;

// Custom Exception
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}

// Bank Account class
class BankAccount {
    double balance;

    // Constructor
    BankAccount(double balance) {
        this.balance = balance;
    }

    // Withdraw method
    void withdraw(double amount) throws NoMoneyException {
        if (amount > balance) {
            // throwing custom exception
            throw new NoMoneyException("INSUFFICIENT BALANCE!");
        } else {
            balance -= amount;
            System.out.println("WITHDRAWAL SUCCESSFUL!");
            System.out.println("REMAINING BALANCE: " + balance);
        }
    }
}

// Main class
public class BANKING_SYSTEM {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("ENTER INITIAL BALANCE: ");
        double balance = sc.nextDouble();

        BankAccount acc = new BankAccount(balance);
```

```
        System.out.print("ENTER AMOUNT TO WITHDRAW: ");
        double amount = sc.nextDouble();

        try {
            acc.withdraw(amount);
        }
        catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
            System.out.println("TRANSACTION COMPLETED.");
        }

        sc.close();
    }
}
```

OUTPUT-

```
ENTER INITIAL BALANCE: 100000
ENTER AMOUNT TO WITHDRAW: 50200
WITHDRAWAL SUCCESSFUL!
REMAINING BALANCE: 49800.0
TRANSACTION COMPLETED.
```

```
ENTER INITIAL BALANCE: 50000
ENTER AMOUNT TO WITHDRAW: 62000
Exception: INSUFFICIENT BALANCE!
TRANSACTION COMPLETED.
```

9. Write a program which enables to use a platform by a train one at a time.

```
class Platform {
    synchronized void use(String train) {
        System.out.println(train + " is entering the platform.");
        try { Thread.sleep(2000); } catch (InterruptedException e) {}
        System.out.println(train + " has departed.");
    }
}

public class TrainSystem {
    public static void main(String[] args) {
        Platform p = new Platform();
        Thread t1 = new Thread(() -> p.use("SuperFastExpress A"));
        Thread t2 = new Thread(() -> p.use("Local Passenger B"));
        Thread t3 = new Thread(() -> p.use("InterCity C"));
        t1.start(); t2.start(); t3.start();
    }
```

```
}
```

OUTPUT-

```
Local Passenger B is entering the platform.
Local Passenger B has departed.
SuperFastExpress A is entering the platform.
SuperFastExpress A has departed.
InterCity C is entering the platform.
InterCity C has departed.
```

10. 10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

```java
class Restaurant {
    private boolean orderPlaced = false;

    // Waiter places order
    synchronized void placeOrder(String order) {
        while (orderPlaced) {
            try {
                wait(); // wait if kitchen is busy
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        System.out.println("Waiter placed order: " + order);
        orderPlaced = true;

        notify(); // notify chef
    }

    // Chef prepares order
    synchronized void prepareOrder() {
        while (!orderPlaced) {
            try {
                wait(); // wait if no order
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        System.out.println("Chef is preparing and delivering order...");
        orderPlaced = false;

        notify(); // notify waiter
    }
}
```

```java
// Waiter Thread
class Waiter extends Thread {
    Restaurant restaurant;

    Waiter(Restaurant r) {
        restaurant = r;
    }

    public void run() {
        String orders[] = {"Pizza", "Burger", "Pasta"};

        for (String order : orders) {
            restaurant.placeOrder(order);

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Restaurant restaurant;

    Chef(Restaurant r) {
        restaurant = r;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            restaurant.prepareOrder();

            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

// Main class
public class RESTURANT {
    public static void main(String[] args) {
```

```
        Restaurant r = new Restaurant();

        Waiter waiter = new Waiter(r);
        Chef chef = new Chef(r);

        waiter.start();
        chef.start();
    }
}
```

OUTPUT-

```
Waiter placed order: Pizza
Chef is preparing and delivering order...
Waiter placed order: Burger
Chef is preparing and delivering order...
Waiter placed order: Pasta
Chef is preparing and delivering order...
```